Dataset with brain atlas A for classification tasks: for the first set of 5 small programming tasks use the Oasis dataset provided by the nilearn package, which can be downloaded using the code below. This is a dataset of structural brain scans (aka voxel-based morphometry) for male and female individuals.

```
import numpy as np

from nilearn import datasets

from nilearn.input_data import NiftiLabelsMasker

from nilearn.image import index_img

import nibabel as nib


brain_data = datasets.fetch_oasis_vbm(n_subjects=100)

yeo = datasets.fetch_atlas_schaefer_2018(n_rois=200)  # this needs to be modified for some tasks

masker = NiftiLabelsMasker(labels_img=yeo.maps, standardize=True, memory='nilearn_cache')

input_variables = masker.fit_transform(brain_data.gray_matter_maps)

output_variable = np.array(brain_data.ext_vars.mf == 'F', dtype=int) # gives 1 for females and 0 for males
```

1. Use sklearn.neighbors.KNeighborsClassifier(n_neighbors=5) and sklearn.linear_model.LogisticRegression(penalty='l2', hyperparameter C set to 1.0) to fit separate predictive models based on 100, 200, 300, …, 900, 1000 brain regions of interest (see 'n_rois' argument for datasets.fetch_atlas_schaefer_2018) based on the first 80% of the 100 structural brain scans to predict sex. Next, evaluate each fitted LogReg / kNN model on the unseen final 20% of the 100 structural brain scans and retrieve classification accuracy in XX.YY%. Please plot the classification accuracy on the test set (x axis=the different atlas resolutions, y axis=prediction accuracy on test set, 2 lines with 2 colors for the 2 different prediction model classes).

2. Use sklearn.neighbors.KNeighborsClassifier() to fit separate predictive kNN models based on 100 brain regions of interest (see 'n_rois' argument for datasets.fetch_atlas_schaefer_2018) using n_neighbors=1,2,3,4,…10 and based on the first 80% of the 100 structural brain scans to predict sex. Next, evaluate each fitted kNN model on the unseen final 20% of the 100 structural brain scans and

retrieve classification accuracy in XX.YY%. Please plot the classification accuracy on the test set (x axis=number of neighbors, y axis=prediction accuracy on test set).

3. Use sklearn.neighbors.KNeighborsClassifier() to fit separate predictive kNN models based on 100 brain regions of interest (see 'n_rois' argument for datasets.fetch_atlas_schaefer_2018) using n_neighbors=1,2,3,4,…10 and based on the full 100% of the 100 structural brain scans to predict sex. Next, evaluate each fitted kNN model on same the 100 structural brain scans and retrieve classification accuracy in XX.YY%. Please plot the classification accuracy on these 100 scans (x axis=the different atlas resolutions, y axis=prediction accuracy). [Think about why kNN shows this performance behavior as a function of its hyperparameter k!]

4. Use KNeighborsClassifier(n_neighbors=3) and LogisticRegression(penalty='l2', C=1.0) to fit separate predictive models using the lowest resolution brain scan data (100 ROIs). Train each model to predict sex based on the first 10, 20, 30, …, 80 of the 100 subjects (you will fit 8 models of each type). Next, evaluate each fitted LogReg / kNN model on the unseen final 20 subjects and calculate classification accuracy. Report these test accuracies in a table to two significant figures (eg. 89.05%). Plot the classification accuracy on the test set (number of training subjects on x-axis (10-80), prediction accuracy on test set on the y-axis; there should be one line for each model and a legend).

5. Use KNeighborsClassifier(n_neighbors=3) and LogisticRegression(penalty='l2', C=1.0) to fit separate predictive models using the lowest resolution brain scan data (100 ROIs). Using the first 80 subjects as the training set, train each model to predict sex based on the first 10, 20, 30,…,100 ROIs (you will fit 10 models of each type). Next, evaluate each fitted LogReg / kNN model on the unseen final 20 subjects and calculate classification accuracy. Report these test accuracies in a table to two significant figures (eg. 89.05%). Plot the classification accuracy on the test set (number of ROIs on x-axis (10-100), prediction accuracy on test set on the y-axis; there should be one line for each model and a legend).

Dataset with brain atlas B for continous prediction tasks: For the second set of 5 small programming tasks use the Oasis dataset provided by the nilearn package, which can be downloaded using the code below. This is a dataset of structural brain scans (aka voxel- based morphometry) for male and female individuals.

```
import numpy as np
from nilearn import datasets
from nilearn.input_data import NiftiLabelsMasker
from nilearn.image import index_img
from sklearn.preprocessing import StandardScaler
import nibabel as nib


brain_data = datasets.fetch_oasis_vbm(n_subjects=100)
crad = datasets.fetch_atlas_craddock_2012()
atlas_nii = index_img(crad['scorr_mean'], 42)


masker = NiftiLabelsMasker(labels_img= atlas_nii, standardize=True)
input_variables = masker.fit_transform(brain_data.gray_matter_maps)
output_variable =
StandardScaler().fit_transform(brain_data.ext_vars.age[:, None])[:,
0] # gives subject age on standard units after z-scoring
```

Note: use Ridge(default parameters) instead of LogisticRegression for the following questions.

6. analogous to task 1, but predicting continuous subject age with sklearn.neighbors.KNeighborsRegressor(n_neighbors=5) and using 43 different versions of Craddock region atlas resolutions (.scorr_mean) instead of the 10 region numbers from Dataset A. Use R2 coefficient of determination instead of classification accuracy, which can be found at sklearn.metrics.r2_score.

7. analogous to task 2, but predicting continuous subject age using KNeighborsRegressor(n_neighbors=1…10). Use crad['scorr_mean'] = 9.

8. analogous to task 3, but predicting continuous subject age with KNeighborsRegressor. Use crad['scorr_mean'] = 9.

9. analogous to task 4, but predicting continuous subject age with KNeighborsRegressor. Use crad['scorr_mean'] = 9.

10. analogous to task 5, but predicting continuous subject age with KNeighborsRegressor. Use crad['scorr_mean'] = 9.